

A Quorum-Based Total Order Broadcast Protocol for Distributed Intelligence Databases

Mattia Rizzo Alan Masutti

Distributed Systems Project – Academic Year 2025–2026

1 Project Structure

1.1 Objective and execution model

The system implements a replicated, fixed-length array of integer positions. Clients may contact any replica; reads are answered locally, whereas a coordinator orders writes. Every client, replica, and emulated point-to-point channel is a Java Akka actor that owns its mutable state and processes one mailbox event at a time. The design assumes static membership, reliable FIFO channels, fail-stop replicas without recovery, failure detection, and a strict majority of correct replicas.

Figure 1 shows the normal request path. An arrow between replicas denotes a dedicated `NetworkChannel` actor for that sender–destination pair. The channel queues messages and adds a controlled random delay, but releases the queue head first; therefore latency never reorders messages sent over the same logical channel.

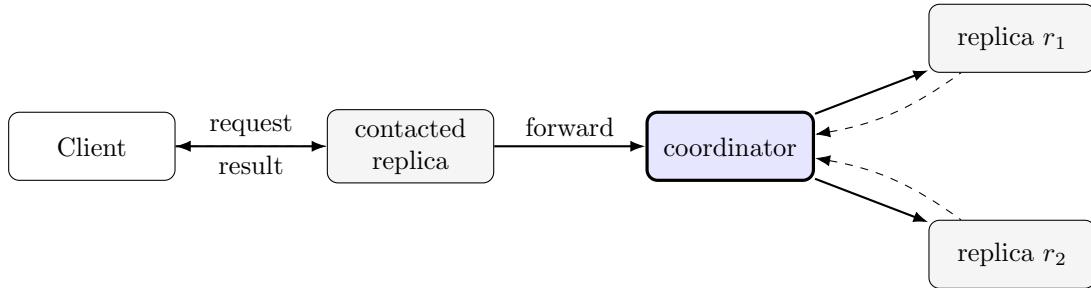


Figure 1: Normal write path: solid arrows carry UPDATE/WRITEOK; dashed reverse arrows carry ACK.

1.2 Transactions as a mean of control and distributed state

A **transaction** is the unit of control of the whole system. Every operation that needs more than one message, such as a read, a write, an update, a heartbeat, or an election, is represented by one transaction object living inside the actor that runs it. The object holds the control part of the operation: the current FSM state, the timeout still pending, the acknowledgements collected so far, and the data of the phase in progress. The actor keeps only the persistent part, that is the array, the latest applied pair, and the history. This split lets a replica run several operations at the same time without mixing their variables, and it keeps each protocol phase independent from the other operation that are concurrent.

The transaction object never leaves its actor; what travels is its `TransactionId`, carried by every `Msg`. Because the identifier pairs the initiator reference with a counter local to that initiator, it is unique in the system without any global allocation. When a message arrives, the actor either recognizes an identifier it already owns and feeds the message to the matching FSM, or creates a new transaction for handling it. The same logical operation therefore exists as one local transaction on each participant, each one in its own phase, and the set of these copies is the distributed state of that operation. The protocol makes progress when a message with a shared identifier moves one more copy forward.

The life of a transaction also acts as a filter. A transaction is registered when it starts and removed as soon as it reaches a terminal state, so a late or duplicated reply arrives when no FSM is waiting for it and is simply dropped. Recovery uses the same rule: after synchronization the replica removes the transactions that belonged to the old coordinator, and every message still in flight for them becomes harmless.

All protocol messages extend from `Msg` class which provide the standard attributes (`transactionId`, `epochPair` and `sender`). The `TransactionId = <initiator, sequence>` correlates a message with one local `Transaction` and is used as routing key, so all the messages sent from the same transaction will be tagged with its id. Heartbeats

reserve transaction sequence 0, ordinary replica FSMs use positive sequences, and elections use negative sequences. The transaction order is the immutable **EpochPair** $\langle e, i \rangle$ and represent the epoch and the update with the sender knows at the sending moment.

2 System Design

2.1 Client operations and the normal update protocol

The client maintains one **currentTransaction** and a FIFO queue. A new operation starts only after the previous one returns or reaches its terminal timeout. This is to preserve the client ordering, in particular when it issue write and reads concurrently.

To perform a **READ**, the client issues a new **ReadTransaction** that sends **ReadMsg**(*idx*) to the selected replica and schedules a local timeout. The replica validates the index through **getPosition**, reads its current array entry, and immediately returns **ReadResultMsg**. The result or the timeout completes the transaction and starts the next queued request.

To perform a **WRITE** $w = (idx, val)$, the client starts a new **WriteTransaction** which takes care of contacting the selected replica by sending a **WriteMsg** that contains the index and the value to be setted. By receiving the message the replica starts the **WriteTransaction** maintaining the same **TransactionId** of the client and when it starts, a new **UpdateTransaction** is issued on the replica by the **WriteTransaction** and the two transactions are linked together.

The **UPDATE** state machine follows these steps:

1. A non-coordinator forwards w to its known coordinator and waits for **UpdateMsg**. If the contacted replica is the coordinator, it enters the coordinator path directly.
2. The coordinator reserves one fresh pair $\langle e, i \rangle$ in mailbox order and broadcasts **UPDATE**($\langle e, i \rangle, idx, val$). The allocator is separate from the latest applied pair, so an assigned but uncommitted operation is not presented as committed state.
3. Each receiving replica creates or advances the matching **UpdateTransaction**, returns **UpdateAckMsg**($\langle e, i \rangle$), and waits for the decision. The coordinator counts itself and waits for

$$|Q| = \left\lfloor \frac{N}{2} \right\rfloor + 1.$$

4. On the first quorum, the coordinator broadcasts **WriteOkMsg** carrying the same non-null pair. A replica applies $P[idx] \leftarrow val$ only after this message, advances its latest pair, stores the update in history, and invokes the required callback.
5. The trigger replica completes its parent write transaction and returns success to the client. Duplicate or stale acknowledgements cannot create a second decision because the coordinator FSM leaves **WAITING_ACK** after its first quorum.

2.2 Why all replicas obtain one total order

Pairs are ordered by the following rule:

$$\langle e, i \rangle < \langle e', i' \rangle \iff e < e' \vee (e = e' \wedge i < i').$$

Within an epoch, the coordinator actor reserves pairs and sends **UPDATE**s in mailbox order. Consider updates A and B with A reserved first. Every participant receives **UPDATE** A before **UPDATE** B on its coordinator-to-replica FIFO channel, and sends **ACK** A before **ACK** B on its return channel. If B has acknowledgements from a quorum, those same actors have already sent their acknowledgements for A ; thus A reaches a quorum no later than B . The coordinator sends **WRITEOK** A no later than **WRITEOK** B , and each coordinator-to-replica FIFO channel preserves this order. A replica can temporarily store prefix $[A]$ while another stores $[A, B]$, but it cannot apply $[B, A]$ or $[B]$. This establishes total-order delivery for every completed epoch.

Sequential consistency then follows from two composition rules. First, the client queue preserves program order. Second, a successful write result is emitted only after the contacted replica applies **WRITEOK**. Therefore, if

the same client next reads that replica, the read is sent after application and observes the write or a later value in the same total order. **Two clients reading at the same time can hit replicas that are not equally up to date, which sequential consistency allows; this never makes two replicas disagree on the order of the writes.**

The protocol also satisfies integrity in the normal case: an update is created from a received request, keyed by one transaction ID and one epoch pair, and applied at most once when that FSM accepts the matching WRITEOK. Validity and eventual delivery depend on the stated reliable-channel, bounded-delay, and surviving-quorum assumptions.

2.3 Timeouts, heartbeat, and crash detection

Timeouts are self-messages scheduled through the Akka scheduler. Each **Transaction** stores its current **Cancellable** and a valid response cancels it. Cancellation alone is insufficient because an expired event may already be in the mailbox, so protocols additionally reject stale events by state or generation.

Detector	Bound used	Reaction
Client READ/WRITE UPDATE phase	Configured public-API deadline $4L$	Calling the proper callback The waiting update is parked until an election is over, it does not start one.
Heartbeat watchdog	$3 \cdot \text{beatInterval} + L$	The follower accepts only the current watchdog generation and requests election once.
Election ACK	$2L$	Mark the unresponsive successor unavailable and retry at the next ring member.

where:

$$L = \text{maxLatency} + \frac{N}{2} \text{maxLatency}$$

The coordinator periodically schedules a local **HeartbeatTickMsg** which cause a broadcasts of **HeartbeatMsg**, followers a check on the receiving of this tick. A **watchdogVersion** increments at every reset: if generation k was cancelled after its event entered the mailbox, **WatchdogExpired**(k) cannot invalidate generation $k + 1$. The heartbeat transaction then enters **ELECTION_REQUESTED**, preventing repeated election requests from the same detector.

Three independent observations can expose a failed coordinator: no UPDATE after a forwarded request, no WRITEOK after UPDATE, or no heartbeat. Their expirations need not be simultaneous. To stop these detectors from creating competing coordinators, election initiation is divided into deterministic slots based on ring distance from the failed coordinator. Its immediate correct successor starts first. A later candidate waits for detector skew plus a complete token/ACK ring traversal for every preceding position. In addition, a replica retains at most one preferred election transaction for a failed coordinator and rejects a competing token with a lower priority.

Crash emulation is part of the protocol boundary, so each replica has a **NONE**, **PENDING** or **CRASHED** status. What matters for the protocol is when the status can change: the check happens once per message sent or handled, never inside a broadcast. A replica therefore either sends a broadcast to the whole group or dies before starting it, and no decision can reach only a part of the replicas.

2.4 Ring election and termination

Replica IDs sorted increasingly define the logical ring. An **ELECTION** token carries an immutable list of candidates. Each candidate contains its replica ID and latest applied **EpochPair**; comparison first selects the greatest pair and then the greatest replica ID to break a tie. This chooses the surviving replica that applied the most recent update, rather than merely the largest node identifier.

On first receiving the token, a participant appends its snapshot, forwards to the next available ring member, acknowledges the previous sender, and waits for its own successor's ACK. When an ACK times out, **RingNavigation** skips that target. Each attempt has an **attemptVersion**, so an old ACK timeout cannot skip the target of a newer attempt. If the best candidate becomes unavailable, it is removed and the next-best candidate is tried. Because each timeout permanently adds one failed member to a finite unavailable set, and a strict majority remains correct, forwarding either reaches a correct member or reduces the candidate set; it cannot wait forever on one edge.

2.5 Synchronization and interrupted updates

The token stops when it reaches a replica that is already in the candidate list. That replica does not elect itself, it forwards the list straight to the best candidate, and the election ends when the token gets there.

The winner opens epoch $e_{new} = 1 + \max_r epoch(r)$, takes $\langle e_{new}, 0 \rangle$ as its own pair, records itself as coordinator and sends the other replicas a **SynchronizationMsg** with the new coordinator ID, the new pair and a copy of its array. A follower accepts it only if the sender is the announced coordinator and belongs to the static group, the failed coordinator is the same it is handling, the new pair is strictly greater than its own and the array has the expected length. It then installs the array, adopts the new coordinator and epoch, and restarts the heartbeat with the ID $\langle coordinator, 0 \rangle$. We move the whole array instead of replaying the missing history entries because its size is fixed, so a single message repairs a replica that lost any number of decisions. Synchronization is sent before the waiting updates are resumed, and both use the same FIFO channel, so a follower never sees a new epoch **UpdateMsg** before the snapshot.

An update whose **UpdateMsg** or **WriteOkMsg** timer expires is not failed, it moves to **WAITING_ELECTION** and waits there. After synchronization every parked transaction that comes from a client write is restarted towards the new coordinator, keeping its **TransactionId** so that the linked **WriteTransaction** can still answer the client. When the new **UpdateMsg** reaches a replica that still holds the parked transaction with that **TransactionId**, the old one is dropped and replaced, so the acknowledgements go to the transaction the new coordinator is waiting for. The retry does not look at how far the update had gone before the crash, and it is correct only because a **WRITEOK** reaches either everybody or nobody: if the coordinator died before it, no replica had applied the update, and if it died after it, no timer expires at all. A crash able to interrupt a broadcast would break this, because the same write would be committed twice under two different pairs, and neither the history nor the active transactions are indexed by **TransactionId**.

2.6 Safety argument and explicit assumptions

All the coordinator decisions form one sequence S . A correct replica has applied an initial part of it and nothing else: it never skips a decision and never applies two of them in the wrong order. During normal operation this holds because a decision needs a quorum and because **WRITEOKs** travel on FIFO channels. An election keeps it: the winner is the replica that got the furthest, it copies its array to everybody and only then opens a greater epoch, so a crash never undoes or reorders a committed update. Client program order fits in S too, since a client waits for one operation before starting the next.

The implementation relies on the specification's static unique IDs, reliable FIFO channels, bounded accurate timeouts, fail-stop crashes without recovery, and a strict majority of correct replicas. Client deadlines must cover detection, election, and synchronization; an arbitrarily short external deadline could let a client advance while an earlier outcome is unknown.

One point is worth stating clearly. The election ranks replicas by the last *applied* pair and ignores the updates that a quorum has already acknowledged, so a decision that nobody applied yet disappears with the coordinator. In our fault model this is harmless, because a **WRITEOK** is delivered to the whole group or to no one, and the write is simply retried in the new epoch. On a network that can deliver a decision to a part of the group only, it would not be: the token would have to carry the pending acknowledgements and the new coordinator would have to close those decisions before opening its epoch. That is the distance between what we implement and uniform agreement in its strongest form.

3 Implementation and Evaluation

3.1 State-machine organization

The actors and the protocol state are kept apart. **Client** and **Replica** create the transactions and route the messages to them, while each **Transaction** has its own **start()**, **computeState(Msg)** and a small explicit FSM. Timers and phase data stay inside the transaction, so a replica can run several transactions at the same time without mixing them up.

FSM	States	How it ends
READ	INIT, WAITING_RESULT	The result arrives, or the timeout reports a failure.
WRITE (client)	INIT, WAITING_RESULT	Result or timeout, then the callback runs and the queue moves on.
WRITE (replica)	INIT, WAITING_UPDATE	The child UPDATE commits and the result goes back to the client.
UPDATE	WAITING_UPDATE, WAITING_ACK, WAITING_WRITEOK, WAITING_ELECTION	The matching WRITEOK applies the value once; a timeout parks the update until an election is over.
HEARTBEAT	COORDINATOR, WATCHING, ELECTION_REQUESTED	The coordinator ticks; a follower that stops seeing the beats asks for an election.
ELECTION	NEW, PARTICIPATING, ELECTED, SYNCHRONIZING	The winner sends its snapshot; a rejection or a synchronization closes the older FSMs.

Dispatch works in two steps. The messages that open a transaction (READ, WRITE, UPDATE and election tokens) are handled by the actor itself, everything else is routed by `TransactionId` to the transaction that owns it. The active transactions are iterated on a copy, because the last transition of a transaction removes it from the list. A replica never sends anything directly: `unicast` and `broadcast` go through one `NetworkChannel` per destination, replies to the clients included, and that is where FIFO order and random latency come from.

Two counters do two different jobs. The transaction counter only gives local IDs that cannot collide, so that messages can be routed. `reserveNextUpdateEpochPair()` decides the database order instead: only the coordinator calls it, and it moves the sequence forward as soon as an update is assigned, while `Replica.epochPair` moves only when a WRITEOK is applied. After a synchronization the reservation restarts from the new epoch. The same pair travels with UPDATE, ACK, WRITEOK, the local state, the history and `WriteFinishMsg`; a message whose pair does not match is ignored, and a null pair is refused wherever the ordering identity is needed.

3.2 Observability and controlled faults

All required externally visible events use the provided `Logger` and callback APIs, so test probes can observe reads, writes, applied updates, election starts, and elected coordinators without shared mutable state. Official logging is timestamped by the template utility; no protocol class prints directly to standard output. Crash instructions are immutable API messages and are interpreted inside the target actor. This makes a failure reproducible at protocol boundaries while preserving normal actor isolation.

Crashes are the only faults we inject, and a `Crash` message can do it in two ways. It either stops the replica at once, or arms a counter on one category of messages: heartbeat, update, WRITEOK or election. The counter is checked once per message sent or handled, so the replica always dies between two messages and never in the middle of a broadcast. Once it is down it drops what it receives and its send helpers return without doing anything. The armed form is what makes recovery reproducible: stopping the coordinator right after it broadcasts an UPDATE leaves the followers waiting for a decision that will never come, so the same run exercises the phase timeouts, the heartbeat detector, the ring election, the state transfer and the retry of the interrupted write.

3.3 Verification strategy and results

Tests are written on two levels. The lower one takes a single mechanism and drives it directly with Akka `TestKit`: message equality and routing by `TransactionId`, the read and the write with their timeouts, the quorum of an update, the heartbeat generations, ring navigation with an unresponsive successor, candidate ordering and the checks that reject an invalid synchronization. The upper one starts a real group of replicas and clients and only looks at what comes out of the callbacks, without touching the internal state, which is also how the tests provided with the assignment work.

The interesting cases live on the upper level. One test writes and reads from a single client and checks that it sees its own operations in program order. Another one has two writers and two readers going at the same time while a replica is killed in the middle of the workload, and checks that each writer keeps its own order inside the sequence the replicas apply, and that no reader ever goes backwards. A third one leaves exactly three replicas out of five alive, the smallest majority, and checks that reads and writes still go through. The last one arms the coordinator to crash right after it broadcasts an UPDATE, then queues two writes from the same client: the surviving replicas must agree on the new coordinator, both values must be applied everywhere, and a later read on a different replica must return the second one. This is the test that covers the whole recovery path end to end.

3.4 Requirement coverage

The requirements of the specification map onto the mechanisms described so far:

- **Total order and reliable delivery.** The coordinator assigns the pairs, a majority has to acknowledge, the value is applied only on WRITEOK, and the channels keep FIFO order.
- **Sequential observations.** A client runs one transaction at a time and queues the others, and a write is reported only after the contacted replica applied it.
- **Crash detection.** The heartbeat watchdog, with a generation number so that an old expiration cannot invalidate the current one.
- **Election with crashes.** Sorted IDs, an immutable token, one ACK timeout per hop, and members that never answer are skipped and never tried again.
- **Most up to date coordinator.** The last applied pair decides who wins, the replica ID only breaks a tie, and the new epoch starts strictly above every pair seen in the token.
- **Convergence and encapsulation.** The whole array is installed before any message of the new epoch, the state belongs to the actor alone, and it leaves it only as a copy.

3.5 Use of artificial intelligence

OpenAI Codex, Claude and GitHub Copilot were used to assist in writing code and text. Every suggestion was read, tested and, where needed, rewritten by us, so the design, the code, the report and the oral discussion remain our own responsibility.

3.6 Conclusion

The whole system rests on few ideas: a quorum decides, the coordinator names every decision with an `EpochPair`, the channels keep FIFO order, and a ring election picks the replica that went the furthest. Most of the design follows from keeping two identifiers apart, the `TransactionId` that says which local FSM a message belongs to and the `EpochPair` that says which replicated decision it carries.

Writing one transaction per operation was the choice that paid off the most, because a replica can follow several updates at once without any shared phase variable, and because a test can drive one FSM alone. What we would do differently is the recovery. Resending the interrupted write is enough for the crashes we can inject, but a coordinator able to die in the middle of a broadcast would need the pending acknowledgements to travel inside the election token, and the updates to be recognised by their `TransactionId`. Together with a synchronization that repairs the history and not only the array, that is where we would go next.